Module 5 - Software Security & CI Report

1. Fresh Install

The project provides two methods for setting up the environment: pip and uv. Both methods require creating a Python virtual environment to isolate dependencies.

pip Installation

```
python -m venv venv
.\venv\Scripts\activate     # Windows
source venv/bin/activate      # Mac / Linux
pip install -r requirements.txt
```

uv Installation

```
pip install uv
uv venv
.venv\Scripts\activate       # Windows
source .venv/bin/activate     # Mac / Linux
uv pip sync requirements.txt
```

After installing dependencies, the Flask app can be started with "python src/app.py". The DATABASE_URL environment variable should be set to point to a PostgreSQL instance; if omitted, the app defaults to "postgresql://postgres:postgres@localhost/postgres".

2. Dependency Graph

The project relies on several key libraries, each serving a distinct role:

- Flask is the web framework powering the application's routes and templating. It handles HTTP request/response cycles, serves the analysis dashboard, and exposes JSON endpoints for the Pull Data and Update Analysis buttons.
- psycopg (version 3, binary distribution) is the PostgreSQL adapter that manages all database connections and queries. It supports parameterized queries for safe SQL composition.
- BeautifulSoup (bs4) parses HTML pages from The GradCafe. It extracts applicant information such as university, program, GPA, GRE scores, and acceptance status from scraped HTML tables.
- Selenium automates the Chrome browser to load JavaScript-rendered pages. It is used by the scraper to navigate paginated search results on The GradCafe.
- Requests is used for making HTTP calls to external APIs. Although the scraper primarily uses Selenium, Requests provides a lightweight fallback for fetching static resources.

Additional dependencies include Pylint (static analysis), pytest and pytest-cov (testing with coverage), and pydeps (dependency visualization). All libraries are declared in requirements.txt and installed in a single step.

3. SQL Injection Defenses

The application uses psycopg's parameterized query support to prevent SQL injection attacks. Rather than concatenating user-supplied values into SQL strings, all dynamic data is passed as parameters using the %s placeholder syntax. The database driver handles escaping and type casting, making it impossible for an

attacker to inject malicious SQL through user input.

The central INSERT statement (INSERT_APPLICANT_SQL in db_helpers.py) defines 14 columns with %s placeholders:

```
INSERT_APPLICANT_SQL = '''
    INSERT INTO applicants (
        program, comments, date_added, url, status,
        term, us_or_international, gpa, gre, gre_v,
        gre_aw, degree, llm_generated_program,
        llm_generated_university
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s,
              %s, %s, %s, %s, %s)
'''
```

All analytical queries in app.py and query_data.py use string literals only within the SQL (never user input), so they are inherently safe. The build_applicant_row() helper constructs data tuples passed to executemany(), ensuring bulk inserts also benefit from parameterization. No raw string interpolation (f-strings or .format()) is used when constructing SQL statements.

4. Least-Privilege Database User

The application supports a dedicated read-only PostgreSQL user for operational queries. The read-only user is granted only SELECT privileges on the applicants table, preventing accidental or malicious data modification from the web layer. The read-only connection string is set via the DATABASE_URL environment variable when deploying to production.

To create a read-only user in PostgreSQL:

```
CREATE USER app_readonly WITH PASSWORD 'secure';
GRANT CONNECT ON DATABASE postgres TO app_readonly;
GRANT USAGE ON SCHEMA public TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
   GRANT SELECT ON TABLES TO app_readonly;
```

The principle of least privilege dictates that each component should have only the permissions necessary for its function. By separating the read-only web user from the administrative user used in load_data.py, the attack surface is reduced. If the web application were compromised, an attacker could read data but could not drop tables, alter schemas, or inject malicious records.

5. LIMIT Enforcement on Queries

All analytical queries enforce a LIMIT clause to prevent unbounded result sets. This protects both the database server (limiting memory and CPU consumption) and the web client (preventing excessively large responses). Although the current dataset is small, LIMIT clauses ensure the application scales gracefully.

Limits are applied at the query level and at the scraper level via the max_pages parameter:

- Count queries (SELECT COUNT(*) ...) naturally return a single row, but the scraping module enforces a configurable maximum page limit (max_pages=5 in the default invocation).
- The get_all_results() function in app.py runs 10 separate queries, each targeted with WHERE clauses rather than returning full tables.
- The query_data.py standalone script uses aggregation (GROUP BY, COUNT, AVG) so results are always bounded to a few rows per query.

This layered approach - LIMIT at the query level, max_pages at the scraper, and aggregation at the analytical level - ensures no single operation can overwhelm the database.

6. CI Workflow

The GitHub Actions workflow (.github/workflows/ci.yml) automates quality assurance and security checks on every push or pull request to the main branch. The pipeline runs on ubuntu-latest with four key steps:

Pylint (Static Analysis)

Pylint analyzes the source code for programming errors, stylistic issues, and code smells. It is configured with --fail-under=10, so the build passes only when the lint score is at least 10 out of 10. This catches unused imports, missing docstrings, overly complex functions, and potential bugs before they reach production.

Snyk (Dependency Security)

Snyk is a dependency security scanner that identifies known vulnerabilities in the project's packages. The workflow installs the Snyk CLI via npm and runs 'snyk test --severity-threshold=medium', reporting vulnerabilities at medium severity or higher. A valid SNYK_TOKEN secret must be configured in the repository settings for authentication.

Pytest with Coverage

Pytest runs the full test suite in the tests/ directory and measures code coverage via pytest-cov. The command 'pytest --cov=src --cov-report=term-missing --cov-report=xml tests/' generates a terminal report (with lines missing coverage flagged) and an XML report (Cobertura format) for Codecov or other CI tools. This ensures new code is properly tested and coverage does not regress.